

Lab 10 Tasks

Key Concepts

Before diving into the tasks, let's understand the core components you'll be working with:

1. **sstream (String Streams):**

- **What it is:** The `<sstream>` header provides classes for performing stream-based I/O (input/output) on strings. Think of it like reading from or writing to a file, but instead of a file, your source or destination is a `std::string`.
- **istringstream:** This specific class is used for *reading* from a string. You can put a string into an `istringstream` object and then use standard stream extraction operators (`>>`) or functions like `getline` to parse data out of the string, just like you would from `cin` or an `ifstream`.
- **When do we need it?** When you have a string containing structured data (like a line read from a file with values separated by commas, spaces, or other delimiters) and you need to easily extract and convert individual pieces of that data. It's often more flexible than purely using string manipulation functions like `substr` and `find` for complex parsing, especially when dealing with mixed data types.

2. **stoi (String to Integer):**

- **What it is:** A function from the `<string>` header (specifically C++11 and later). It stands for "string to integer". It takes a `std::string` as input and attempts to convert the numerical characters within that string into an `int` value.
- **When do we need it?** When you have read a string that represents a number (e.g., "123", "45") and you need to perform mathematical operations or comparisons on that number as an integer type.

3. **substr (Substring):**

- **What it is:** A method of the `std::string` class. It allows you to extract a portion (a "substring") of an existing string. You typically specify the starting position (index) of the substring and optionally the number of characters to include.
- **How it's used:** `myString.substr(pos)` extracts the substring starting from index `pos` until the end of the string. `myString.substr(pos, len)` extracts `len` characters starting from index `pos`. Remember that string indices start at 0.

- **When do we need it?** When you need to isolate a specific part of a string based on its position or in combination with `.find` to get text *after* a certain delimiter or marker.

4. `.find`:

- **What it is:** A method of the `std::string` class. It searches for the first occurrence of a specified substring or character within the string.
- **How it's used:** `myString.find(searchValue)` searches for `searchValue` within `myString`. It returns the starting index (position) of the first match.
- **Return Value:** If the `searchValue` is found, it returns a `size_t` value representing the starting index. If the `searchValue` is *not* found, it returns a special value called `std::string::npos`.
- **`std::string::npos`:** This is a constant value defined in the `std::string` class, representing "no position". It's a reliable way to check if `.find` failed to locate the target. It's typically the largest possible value for `size_t`.
- **When do we need it?** When you need to locate the position of a delimiter, a keyword, or a specific sequence of characters within a string so you can then extract parts of the string relative to that position using `substr`.

5. `size_t`:

- **What it is:** An unsigned integer type provided by the C++ standard library. It's guaranteed to be large enough to represent the size of any object in memory or the maximum number of elements in any container.
- **When is it used with strings?** `size_t` is the type returned by string methods like `.size()` (which gives the length of the string) and `.find()` (which gives the position). It's also the type expected for indices and lengths in methods like `substr()`. Using `size_t` for string sizes and positions is good practice as it correctly reflects the nature of these values (they are always non-negative) and avoids potential issues with implicit conversions between signed and unsigned integers.

Tasks

vehicles.txt

```
// Vehicle Database
AutonomousCar,AC101,RoboTaxi,2023,SW:4.5,CERT-AI
ElectricVehicle,EV202,VoltZ,2022,Battery:150,CERT-ECO

HybridTruck,HT303,GreenHauler,2021,Cargo:5000|Battery:75,CERT-HYBRID
```

Task 01: Reading the Data File

- Open the provided data file (named "vehicles.txt") and read through it line by line.
- **Task:** Write C++ code to declare an input file stream object (ifstream), open the specified file. Include a check to make sure the file opened successfully. If it fails, print an error message. Then, use a loop with getline to read each full line from the file into a std::string variable. (For simplicity, you can skip comment lines starting with '#' and empty lines if desired, but the core task is opening and reading line by line).
- **Concepts Covered:** File I/O (ifstream, open, is_open, getline), std::string.

Task 02: Basic Data Extraction with sstream

- Take a single line that has been read from the file and split it into its main parts using sstream.
- **Task:** Inside the loop from Scenario 1, after reading a line into a std::string variable, create an istream object initialized with that line. Use getline repeatedly on the istream with the comma character (,) as the delimiter to extract the values for Type, ID, Name, Year (as a string), ExtraData (as a string), and Certification into separate std::string variables. Print these extracted string values to verify your parsing.
- **Concepts Covered:** sstream (istream), getline (with sstream and delimiter), std::string.

Task 03: Converting String Data to Numbers

- **Goal:** Convert the string representation of the vehicle's year into an integer.
- **Task:** Take the `std::string` variable holding the Year value (extracted in Scenario 2). Use the `stoi` function to convert this string into an integer. Declare an `int` variable to store the result. If the conversion fails (though the problem simplifies error handling, mention the function's purpose), `stoi` would normally throw an exception, but for this task, assume valid input for conversion or simply note that `stoi` performs this conversion. Print the resulting integer value.
- **Concepts Covered:** `stoi`, `std::string`, `int`.

Task 04: Parsing Simple ExtraData Formats (AutonomousCar, ElectricVehicle)

- **Goal:** Extract and convert the numerical value from the ExtraData string for the "AutonomousCar" (Software Version) and "ElectricVehicle" (Battery Capacity) types.
- **Task:** Using the Type string variable and the ExtraData string variable (from Scenario 2), write `if` or `else if` statements to check if the Type is "AutonomousCar" or "ElectricVehicle".
 - If "AutonomousCar", use the `find` method to locate the colon (:). Use `substr` starting from the position *after* the colon to extract the number string. Then use `stof` to convert this substring into a floating-point number (`float`) representing the Software Version.
 - If "ElectricVehicle", use `find` to locate the colon (:). Use `substr` starting from the position *after* the colon to extract the number string. Then use `stoi` to convert this substring into an integer representing the Battery Capacity.
 - Print the extracted numerical values along with the vehicle ID. Remember to use `size_t` for positions returned by `find`. Add a check using `std::string::npos` to see if the colon was found before attempting `substr`.
- **Concepts Covered:** `find`, `substr`, `stof`, `stoi`, `size_t`, `std::string`, `if/else if`.

Task 05: Parsing Complex ExtraData Format (HybridTruck) and Type Handling

- **Goal:** Extract and convert both numerical values from the ExtraData string for the "HybridTruck" type, which uses both : and | delimiters, and tie all type checks together.
- **Task:** Add another else if case for the "HybridTruck" type. In this case, the ExtraData format is "Cargo:X|Battery:Y".
 - Use find to locate the colon (:), and find again to locate the pipe symbol (|). Remember these return size_t.
 - Use substr with the correct starting position (after :) and length (up to |) to extract the Cargo value string. Use stoi to convert it to an int.
 - Use substr with the correct starting position (after "Battery:") to extract the Battery value string. Use stoi to convert it to an int.
 - Add a final else block to handle any unknown vehicle Type, printing an error message. Ensure checks for std::string::npos are used for find results before using substr. Print the extracted Cargo and Battery values.
- **Concepts Covered:** find (multiple uses), substr (with position and length), stoi (multiple uses), size_t, std::string, if/else if/else.

Task 06: Storing and Loading Inventory Items (Binary I/O)

- You are developing a simple inventory system. You need to save basic item data (an item ID number and a fixed-length item name) directly to a file and then be able to load it back.
- **Task:** Define a simple C++ structure (or class) named InventoryItem that contains an integer itemID and a character array itemName of size, say, 20 characters. Write a small program that creates an InventoryItem object, writes its data to a binary file named "inventory.dat" using the write() function, closes the file, then opens the same file, reads the data back into a *different* InventoryItem object using the read() function, closes the file, and finally prints the loaded item ID and name.
- **Key Concepts:** read(), write(), Binary file mode (ios::binary), ofstream, ifstream, structure/class definition, fixed-size character arrays.

Task 07: Checking Write Position

- You are logging sensor data to a file. After writing a block of data, you want to know exactly where the next write operation will occur in the file.
- **Task:** Create an output file stream (ofstream) and open a text file (e.g., "sensor_log.txt"). Write a couple of lines of dummy sensor data (just simple strings). After each write operation, use the tellp() function to get the current position of the put pointer and print it to the console. Observe how the position changes as you write more data. Close the file at the end.
- **Key Concepts:** ofstream, open(), tellp(), writing data (<< operator), streampos.

Task 08: Updating a Specific Record (Seek Put Pointer)

- You have a simple configuration file where settings are stored at fixed positions. You need to update just one setting (e.g., a status flag) without rewriting the entire file.
- **Task:** Create a file named "config.txt" and write some initial fixed-length data blocks (e.g., "AAAAA", "BBBBB", "CCCCC"). Close and reopen the file using fstream with both input and output modes (ios::in | ios::out). Use the seekp() function to move the put pointer to the beginning of the second data block (e.g., position 5, assuming each block is 5 characters). Write new data ("XXXXX") of the same length to overwrite the second block. Close the file and then read the entire file back to verify the change.
- **Key Concepts:** fstream, ios::in, ios::out, seekp(), writing data (write() or <<), file positions (absolute).

Task 09: Checking Read Position

- You are reading a large log file piece by piece. After reading a certain amount of data, you want to record exactly how far you've progressed into the file.
- **Task:** Create a text file (e.g., "large_log.txt") with several lines of text. Open it with an input file stream (ifstream). Read the first 10 characters using the read() function (or read a line using getline). Immediately after reading, use the tellg() function to get the current

position of the get pointer and print this position. Read another block of data (e.g., the next 10 characters or another line) and again print the position using `tellg()`. Close the file at the end.

- **Key Concepts:** `ifstream`, `open()`, `read()` (or `getline`), `tellg()`, `streampos`.

Task 10: Skipping Ahead in a File (Seek Get Pointer)

- You have a large file with records, and you only need to read a specific record located deep within the file, without reading all the data before it.
- **Task:** Create a text file (e.g., "data_records.txt") with several distinct lines, each representing a "record" (e.g., "Record 1\n", "Record 2\n", "Record 3\n", "Record 4\n"). Open it with an input file stream (`ifstream`). Use the `seekg()` function to move the get pointer directly to the start of the third record (you'll need to calculate the byte offset based on the lengths of the preceding records, including the newline characters). After seeking, read and print *only* the content of the third record using `getline`. Close the file.
- **Key Concepts:** `ifstream`, `open()`, `seekg()`, `getline`, calculating offsets, `ios::beg`.

Contents for the text files

Task 7: sensor_log.txt

Sensor 1: 25.5 C

Sensor 2: 98.1 %RH

(Note: The file will contain exactly these two lines, including the newline character at the end of each line.)

Task 8: config.txt

(Initial Content before the update)

AAAAABBBBBBCCCCC

(Note: The task involves overwriting the "BBBBB" part using `seekp`, resulting in "AAAAAXXXXXCCCCC" in the file after the program runs, but the initial content written is as shown above.)

Task 9: large_log.txt

This is the first part of the log.

This is the second part of the log.

And the third part.

(Note: The file contains three lines. The last line might or might not have a trailing newline depending on the exact writing method, but the seek calculation in the task assumes the specific lengths as written.)

Task 10: data_records.txt

Record 1

Record 2

Record 3

Record 4

(Note: Each line ends with a newline character, which is crucial for calculating the offsets for seekg.)